香港科技大学(广州)
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY (GUANGZHOU)
信息枢纽
INFORMATION HUB
人工智能学域
ARTIFICIAL INTELLIGENCE

VLA Models I

Course AIAA 4220

Week 12- Lecture 23

Changhao Chen
Assistant Professor
HKUST (GZ)

Project 2 & Project 3 Grading Rubric

The grades for Project 2 and Project 3 consist of two components.
Project 2 is worth **20 points**, and Project 3 is worth **30 points**.

For each project, the **leaderboard** and **presentation** each account for **50%** of the score.

(1) Leaderboard

- **Top-ranked teams receive A**
- **Middle-ranked teams receive B**
- **Teams above the baseline receive C**
- **Teams below the baseline receive D**

(2) Presentation

Evaluated by **Instructor, TA**, and **peer review**.

- **Each student has three peer-review votes**
- **Instructor and TA each will have six votes**
- **Top 10 teams present**
- After voting, all teams are ranked overall

Score multiplier: 1.1x for solo, 1.05x for two people and 1.0x for three people group.

Grades are divided into four tiers:

- **A – 100% credit**
- **B – 80% credit**
- **C – 60% credit**
- **D – Fail (0% credit)**

(3) Additional Requirements

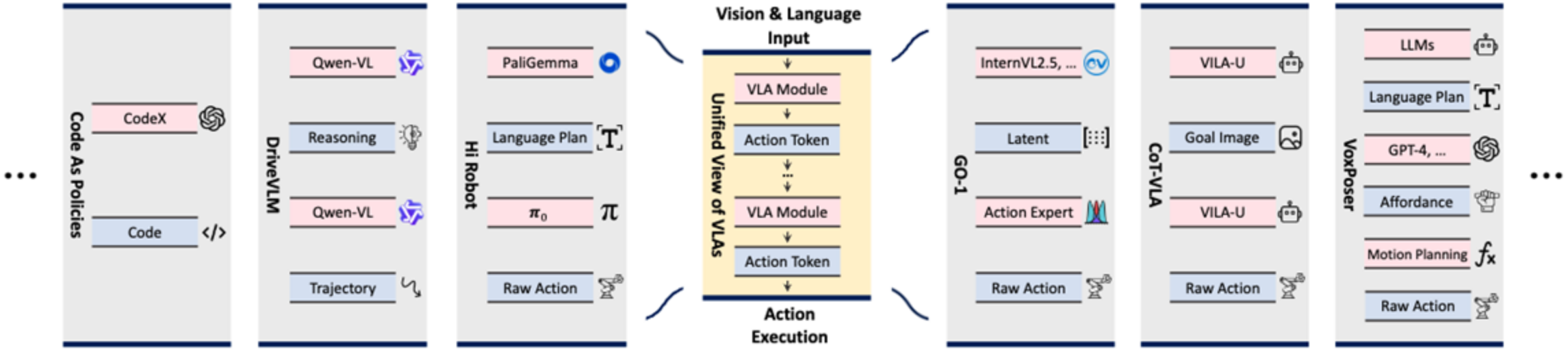
Each student must submit before 23 Dec 2025:

1. **Important code and explanations**
2. **A technical report (1–2 pages)**
3. **Final slides**

Recap: Vision-Language-Models

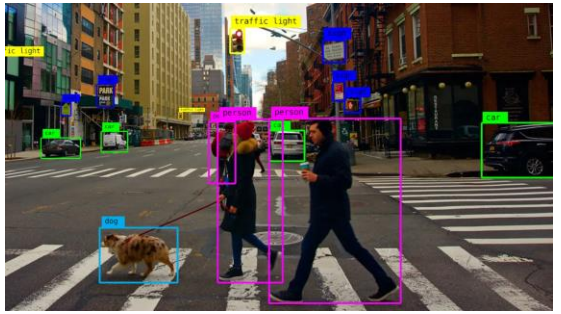
From LLM to VLM to VLA

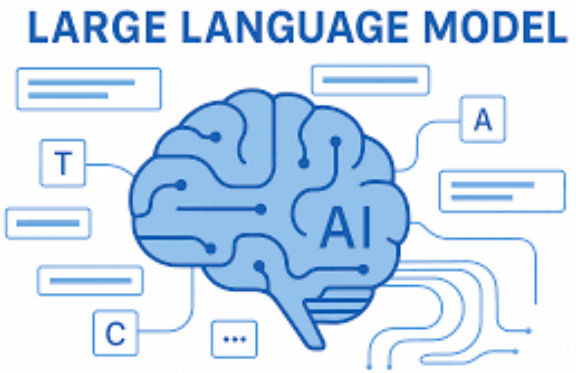
- For embodied AI tasks, the outputs we want are **action tokens**
- The inputs are similar (vision + text tokens) with LLM and VLM
  - Plus robot states inputs
- Tremendous progress in language and multimodal (vision+language) models
- We can leverage these to improve capabilities to learn new things



VLA Models

- Robots must operate in **open-ended, unstructured** environments
- Humans naturally express goals using **language**, not code
- Traditional pipelines require:
  - Vision: object detection, segmentation
  - SLAM/perception
  - Symbolic planning
  - Controllers
- These modules:
  - require heavy engineering
  - often brittle
  - struggle with generalization
- VLA models unify **perception + language understanding + action generation**





VLA Models

Definition:
Models that map (visual observations + language instructions) → sequences of actions

•Inputs: RGB images, video frames, sensor tokens
•Language: natural-language instructions
•Outputs: robot actions (end-effector deltas, gripper states, navigation commands)
•Often implemented as **transformers** or multimodal LLMs

VLA models sit at intersection of **vision**, **NLP**, and **robotics**.

The VLA Problem Formulation

Given:

- I_t: visual inputs (images, depth maps, proprioception)
- L: instruction ("pick up the red mug")

Goal: learn a policy

- $\pi(a_t|I_t, L)$
- where a_t is a continuous or discrete action

Includes both:
•**perception grounding** ("which mug?")
•**semantic understanding** ("red mug", "put into sink")
•**control** ("move arm", "grasp", "navigate")

Typical Applications

Humanoid Systems

The key advantage of VLAs in humanoid systems is their ability to scale across diverse tasks using shared representations. Unlike traditional robotic systems that rely on taskspecific programming or modular pipelines, VLA-powered humanoids operate under a unified token-based framework.

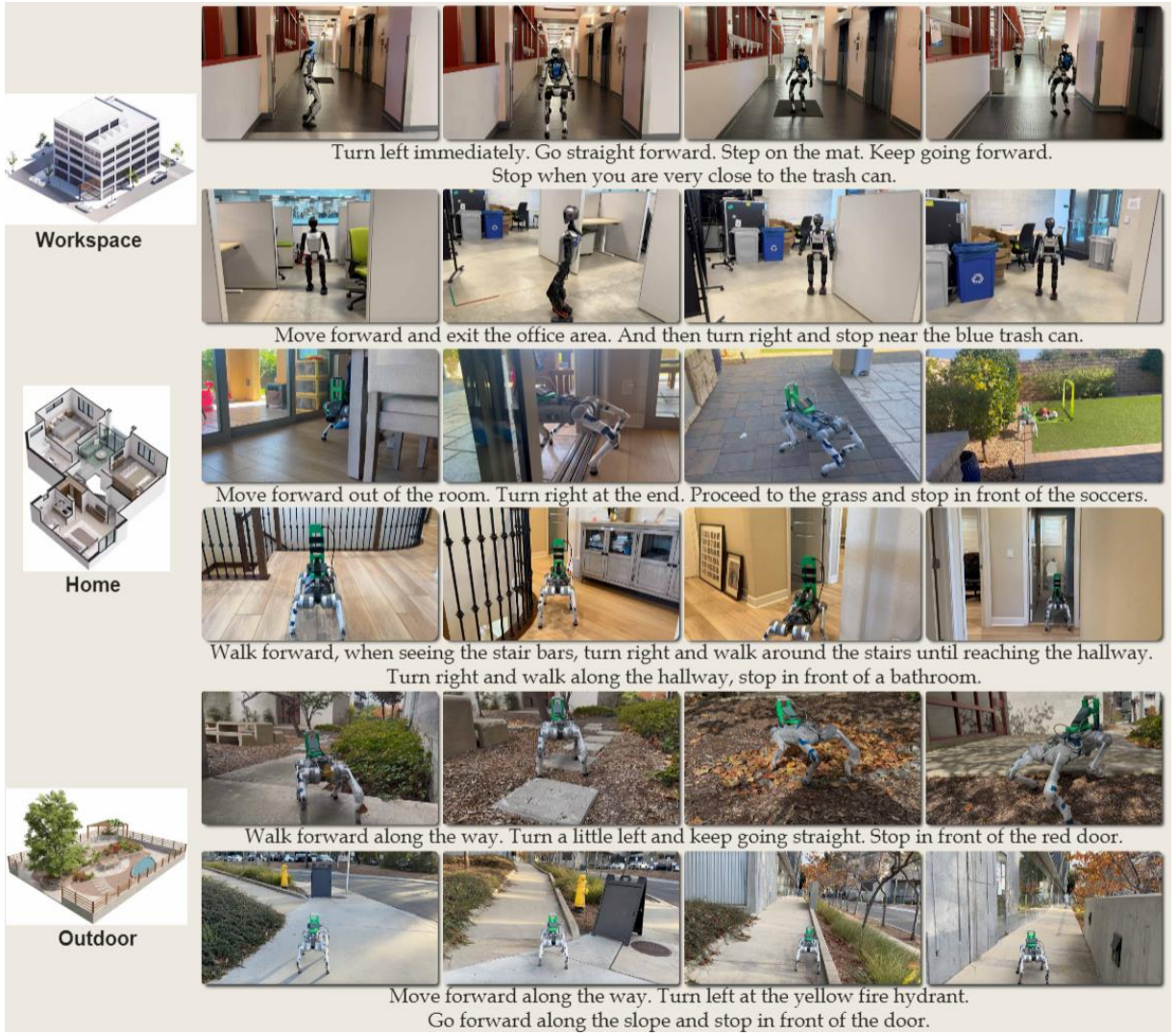
VLA enables humanoid robots to operate effectively in human-centric spaces, such as households, hospitals, and retail environments.

Typical Applications

Vision Language Navigation (VLN)

VLN bridges the gap between visual cues and linguistic instructions. Users can guide systems using natural language combined with real-world images. This concept is vital for applications such as autonomous vehicles, robotics, and augmented reality experiences, where understanding both visual cues and natural language is essential for navigation.

By simply describing or showing a visual clue, the system can guide them to the exact spot, significantly reducing search time and increasing maintenance efficiency. Other uses include assistive technologies for visually impaired people or guiding customers to desired products or stores.

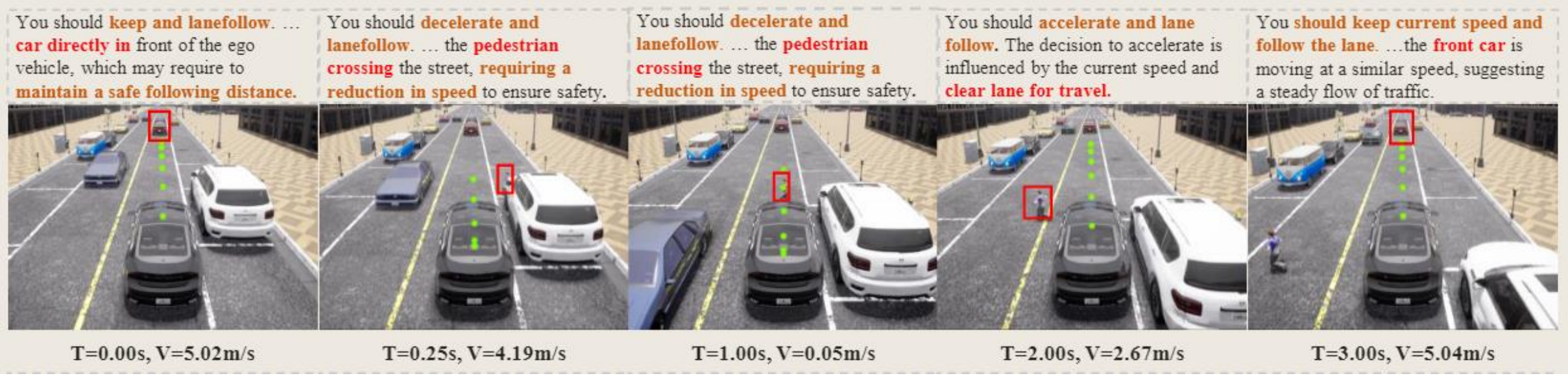


Typical Applications

Autonomous Vehicle Systems

Autonomous vehicles (AVs), including self-driving cars, trucks, and aerial drones, represent a frontier application domain for VLA models, where safety-critical decision-making demands tightly coupled perception, semantic understanding, and real-time action generation.

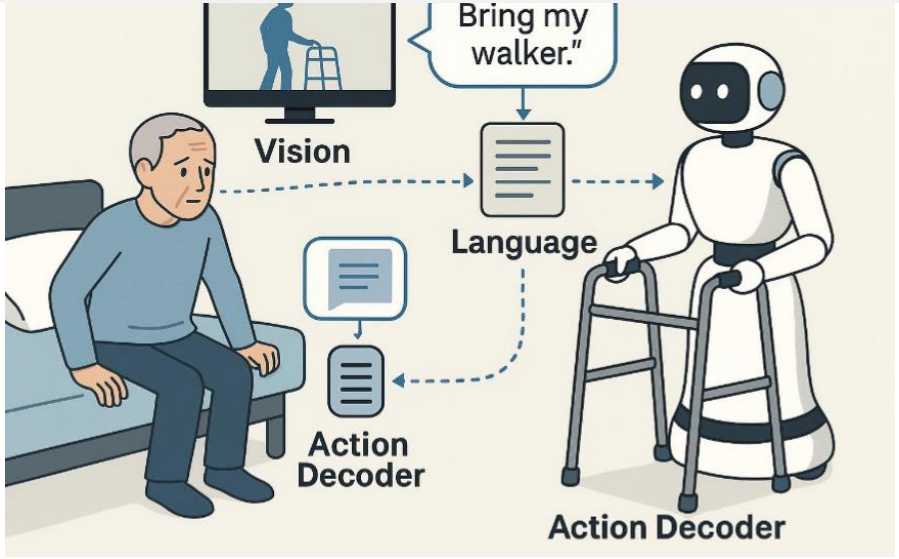
Unlike traditional modular AV pipelines that decouple perception, planning, and control, VLA frameworks offer an integrated architecture that processes multimodal inputs—including visual streams, natural language instructions, and internal state information—within a unified autoregressive model capable of outputting precise control signals.



Typical Applications

Healthcare and Medical Robotics

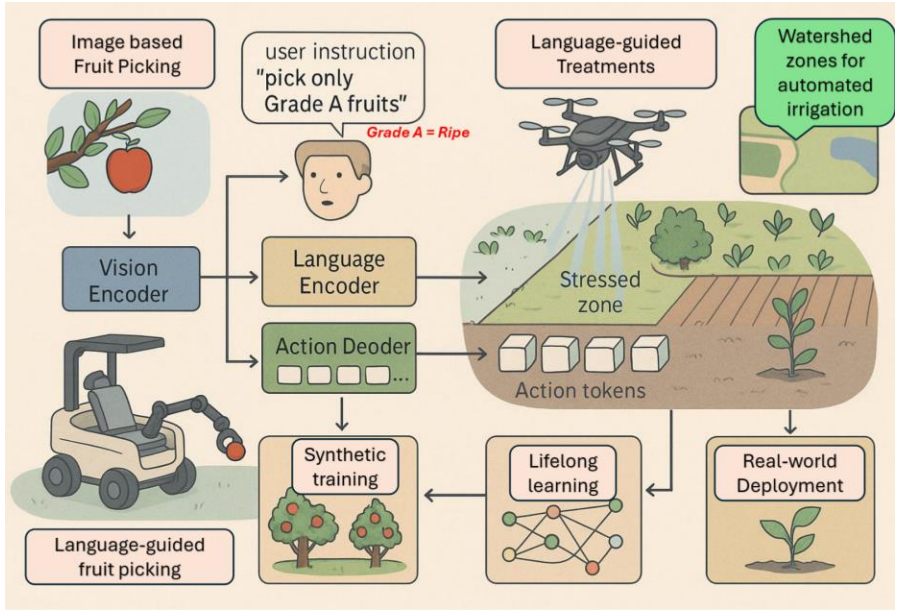
VLA models offer a flexible framework that integrates real-time visual perception, language comprehension, and fine-grained motor control, enabling medical robots to understand high-level instructions and autonomously perform intricate procedures or assistance tasks



Automated Agriculture

VLA models are emerging as transformative tools in precision and automated agriculture, offering intelligent, adaptive solutions for labor-intensive tasks across diverse farming landscapes.

In modern orchards and crop fields, VLAs can process visual inputs from RGB-D cameras, multispectral sensors, or drones to monitor plant growth, detect diseases, and identify nutrient deficiencies.



VLA Model Architecture

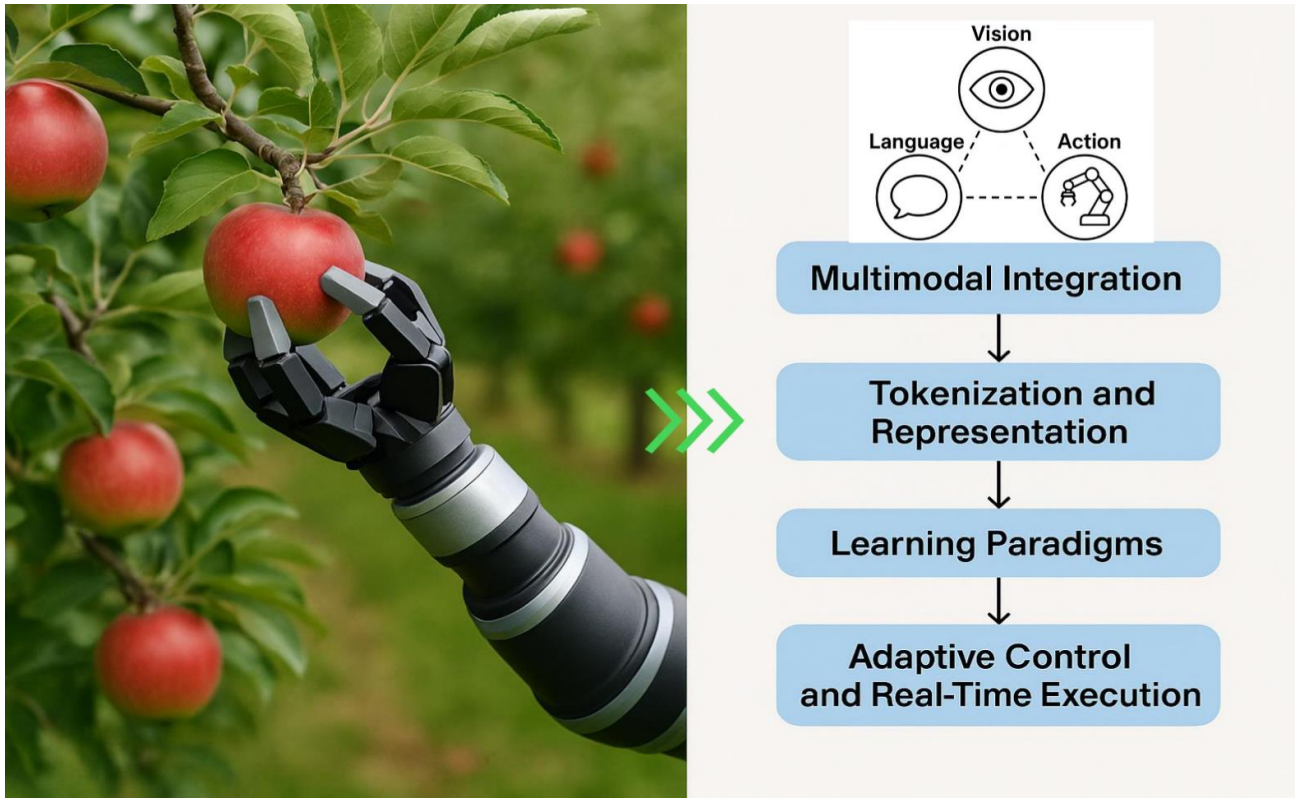
VLA Model Architecture Overview

Components:

1.Vision Encoder
2.Language Encoder
3.Multimodal Fusion Module
4.Action Decoder / Policy Head

Flow:

Image → Vision Tokens
Text → Language Tokens
Fusion → Shared Representation
Action Decoder → Robot Commands



VLA Model Architecture

Visual Encoder

Common architectures:

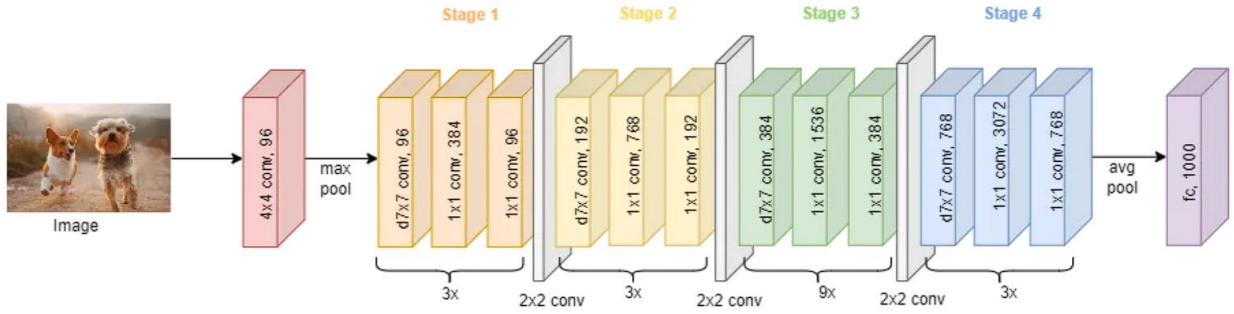
- **CNNs**: ResNet, EfficientNet
- **Vision Transformers (ViT)**
- **Multimodal Encoders**: CLIP, Flamingo-style vision towers

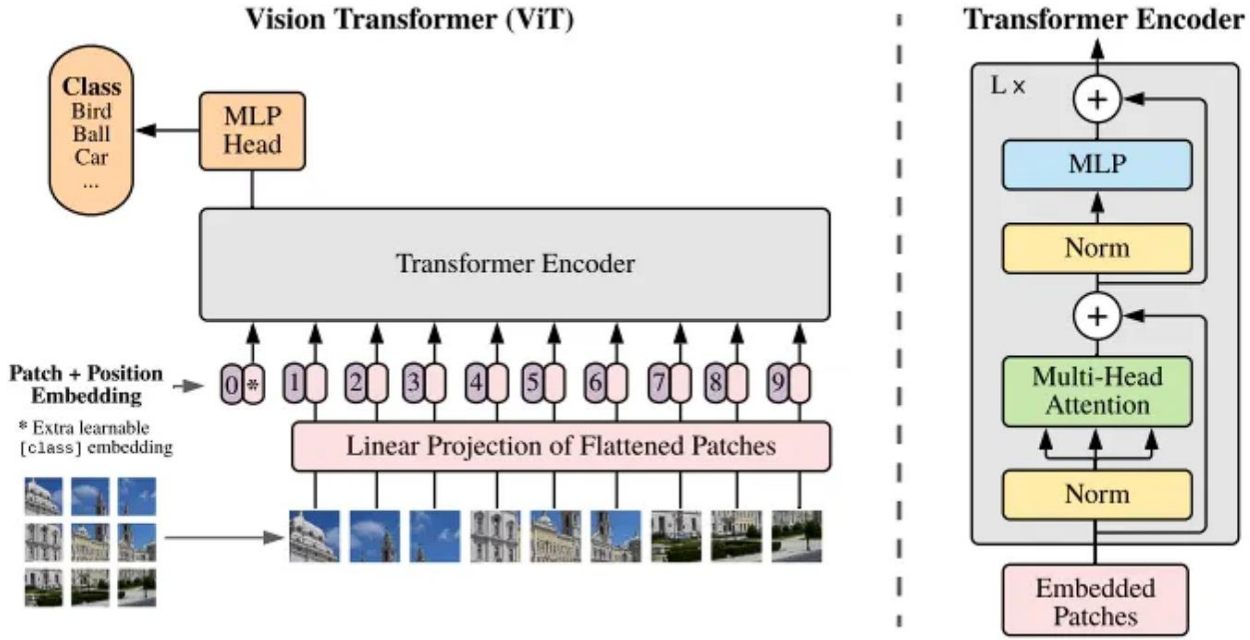
Output formats:

- Fixed-size latent embedding
- Token sequence (ViT patches)

Key abilities:

- Object recognition
- Spatial awareness
- Semantic grounding (“cup”, “on the table”)
- Temporal modeling for actions (using stacked frames)





VLA Model Architecture

Language Encoder

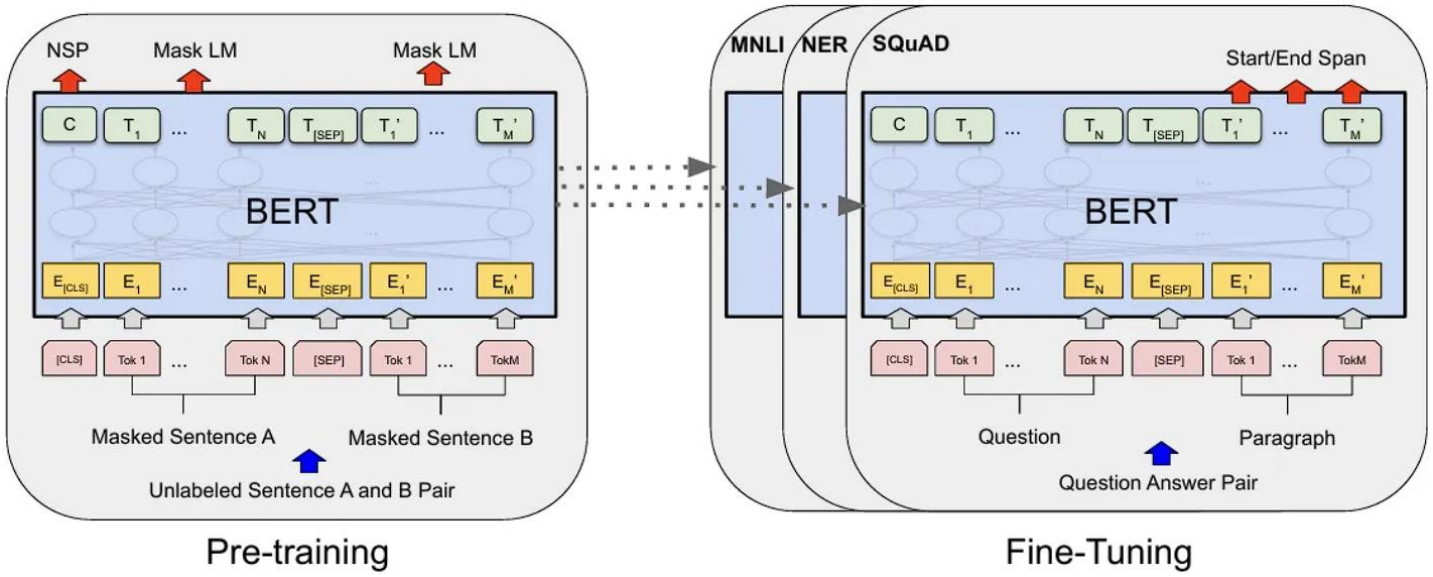
Input: natural-language instruction

Examples:
- "Pick up the red cup."
- "Stack the blocks by size."
- "Navigate to the kitchen."

Typical encoders:
- Transformer LLMs (T5, GPT, PaLM)
- Smaller text transformers for real-time robots

Outputs:
- instruction embedding
- token sequence

Role:
- Provide **task intent**
- Inject **semantic priors**
- Guide perception and action

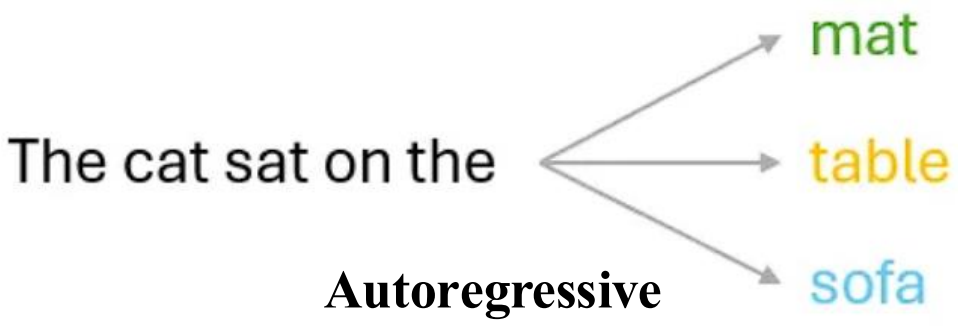


VLA Model Architecture

How Language Controls Robot Behavior

Language encoder helps robots:
- Identify **target objects** using descriptions
- Understand **task constraints**
- Execute **multi-step reasoning**
- Choose **appropriate skills/action primitives**

- Examples:
- "Pick up the big cup" → size-based filtering
- "Carefully open the drawer" → action modulation
- "Move to the chair behind you" → spatial relation reasoning



Autoregressive

Autoencoder

VLA Model Architecture

Multimodal Fusion

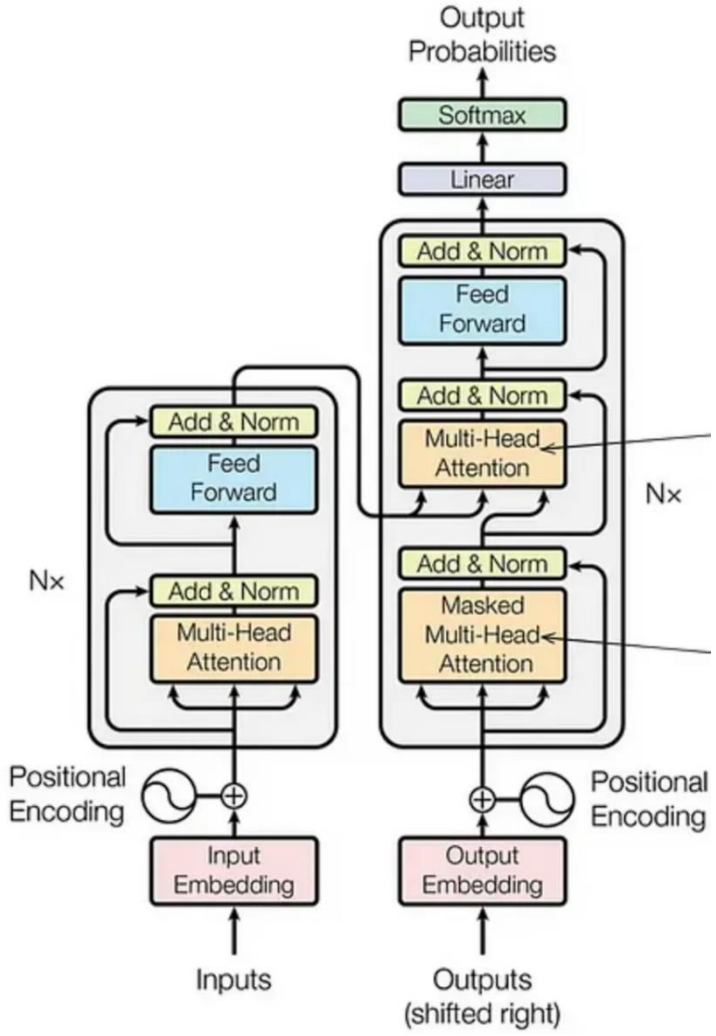
- **Early Fusion**: concatenate vision + language embeddings
- **Late Fusion**: process separately, fuse at final layers
- **Cross-Attention** (common):
  - language tokens attend to vision tokens
  - vision tokens attend to language
- **FiLM Conditioning**: modulate visual features with text features
- **LLM as the central planner** (PaLM-E):
  - vision tokens embedded directly inside LLM
  - LLM interprets scene + task jointly

VLA Model Architecture

Fusion in Transformers

In VLA transformers:
•Vision tokens form a sequence
•Language tokens form another sequence
•Fusion:
- concatenation into a unified token sequence
- or cross-attention between streams

Benefits:
•Scales well
•Allows long-range dependencies
•Enables semantic grounding:
- “red cup” → attention highlights red cup region



For example: Cross-attention mechanisms serve as a fundamental building block for multimodal fusion in modern AI systems, enabling sophisticated information exchange between different modalities such as vision, language, and audio. Features from one modality act as queries while features from another modality serve as keys and values, allowing the model to selectively attend to relevant cross-modal information through learned attention weights.

VLA Model Architecture

Tokenization and Representation

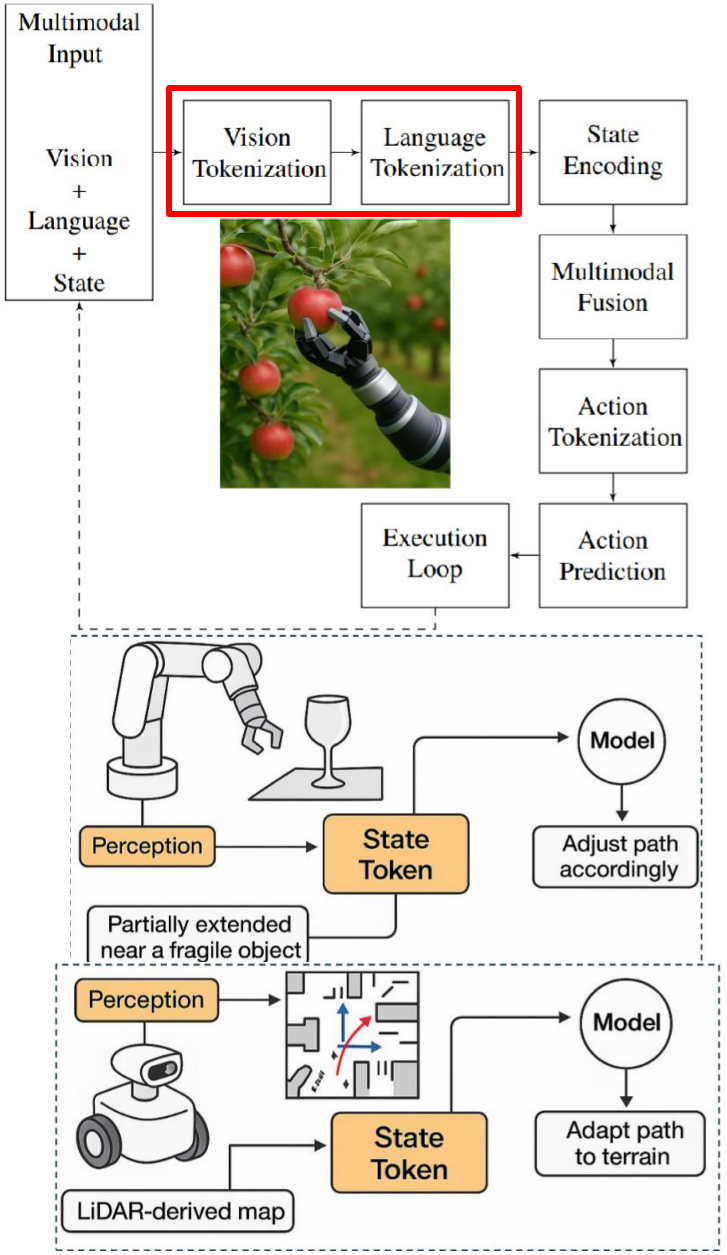
- Inspired by autoregressive generative models like transformers, **modern VLAs encode the world using discrete tokens that unify all modalities—vision, language, state, and action into a shared embedding space.**
- This allows the model to not only understand "what needs to be done" (semantic reasoning), but also "how to do it" (control policy execution) in a fully learnable and compositional way.

Prefix Tokens

Prefix tokens serve as the contextual backbone of VLA models. These tokens encode the environmental scene and the accompanying natural language instruction into compact embeddings that prime the model's internal representations.

State Tokens

State Tokens encode real-time information about the agent's configuration—joint positions, force-torque readings, gripper status, end-effector pose, and even the locations of nearby objects.

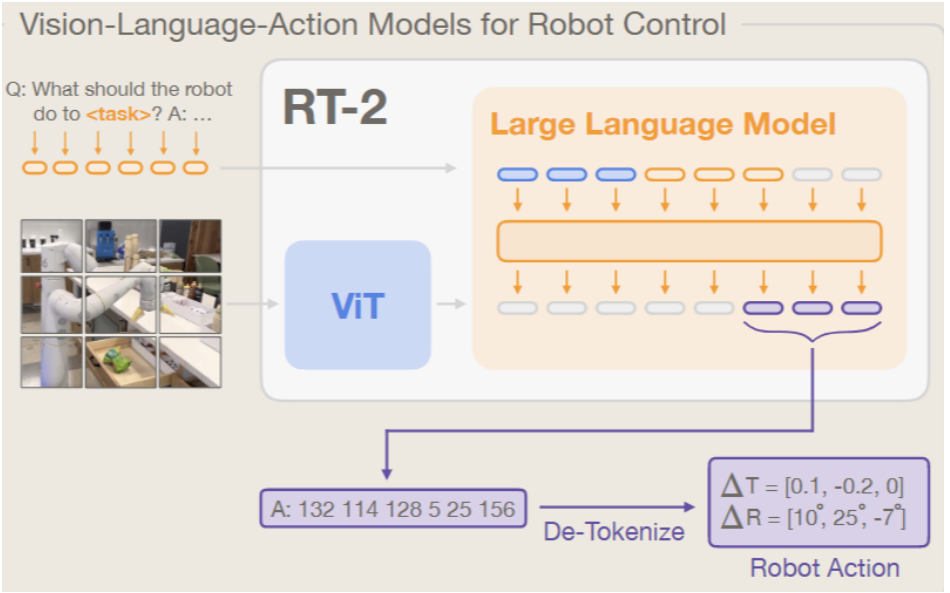


VLA Model Architecture

Action Tokens

The final layer of the VLA token pipeline involves action tokens, which are autoregressively generated by the model to represent the next step in motor control.

- Continuous States Prediction
- Discrete Action Sequence Prediction



VLA Model Architecture

Action Spaces in VLA Models

Low-level actions
•Joint torques
•End-effector velocities
•Gripper open/close
•Continuous control

Mid-level actions
•Spatial waypoints
•Object-centric actions
•Motion primitives

High-level actions
•Skills: grasp, push, pick-and-place
•Symbolic steps (task decomposition)
Different datasets choose different action levels.

VLA Model Architecture

Action Decoders

Often implemented as:
•Transformer decoder (RT-1, RT-2)
•MLP policy head
•Autoregressive token predictor

•Outputs:
•Future action tokens (RT models use discrete tokens)
•Motor commands for the next timestep
•Parameterized skills

Key goal: convert multimodal fused representation → robot behavior.

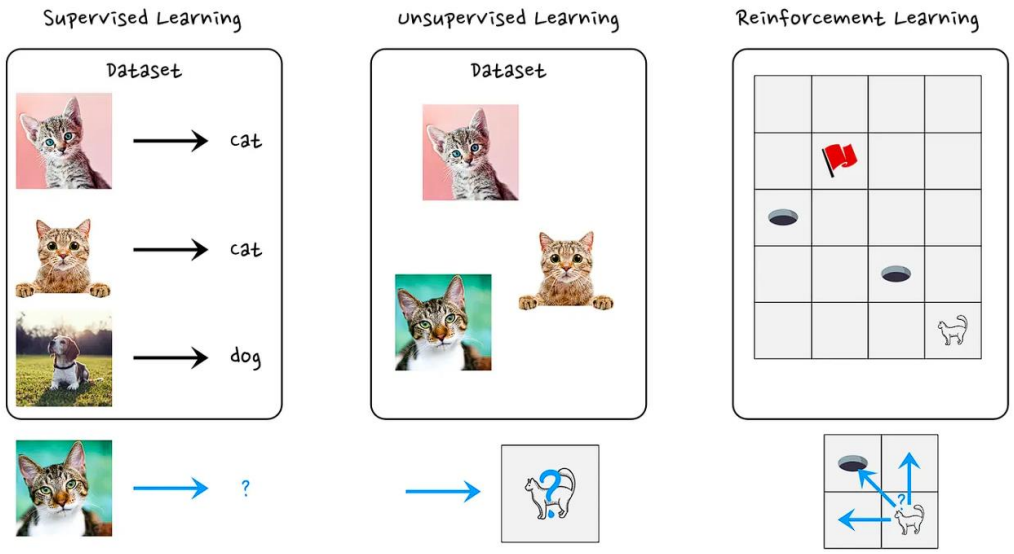
Learning Paradigms

1. Behavior Cloning (Imitation Learning)
•Most common
•Learn mapping from demo trajectories
•Supervised learning:
- minimize error between predicted + expert actions

2. Reinforcement Learning
•Reward-driven behavior
•Harder for long-horizon robotic tasks

3. Self-supervised Representation Learning
•Masked vision modeling
•Contrastive vision–language learning

4. Large-scale Multimodal Pretraining
•Web-scale image–language data improves robot generalization



Learning Paradigms

Imitation Learning - Behavior Cloning

In IL instead of trying to learn from the sparse rewards or manually specifying a reward function, an expert (typically a human) provides us with a set of demonstrations. The agent then tries to learn the optimal policy by following, imitating the expert's decisions.

1.Collect robot demonstration data
2.Synchronize images, text, and actions
3.Train a transformer to minimize action prediction loss
4.Deploy on robot to perform tasks

Advantages:
•Stable
•Scalable
•Works well with many demos

Limitations:
•Struggles with rare events
•Requires high-quality demonstrations

Learning Paradigms

Reinforcement Learning

- At every timestep t, the agent needs to choose an action **a**.
- After this action it might receive a reward **r** and we get a new observation of its state **s**.
- The RL problem is trying to maximize the cumulative reward the agent gets over time.

In the case of RL, we consider an agent that is actively interacting with an environment. Through its interactions, it is possible that it may influence the environment that it operates in.

The "dataset" we need to consider here, are the actions our agent took and the accumulated rewards it got by taking those actions.

Robotic Systems Lab: Legged Robotics at ETH Zürich
https://youtu.be/3jDoPobFgwA

Learning Paradigms

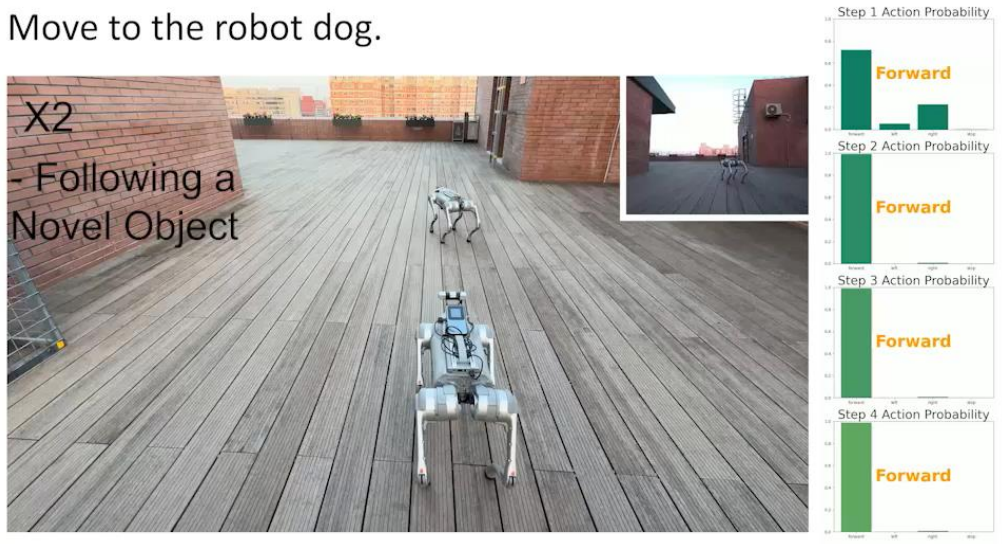
Typical Loss Functions:

- **Action prediction loss** (L2 or cross entropy)
- **Contrastive loss** (vision–language alignment)
- **Sequence modeling loss** (autoregressive)
- **Auxiliary losses**:
  - masked modeling
  - waypoint prediction
  - object grounding

- Sometimes hierarchical losses for multi-step tasks

Adaptive Real-Time Prediction

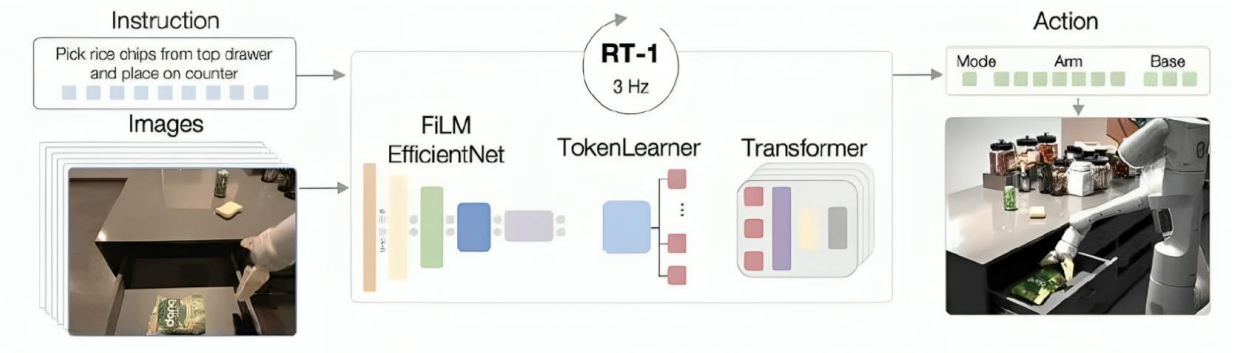
- Another strength of VLAs lies in their ability to **perform adaptive control, using real-time feedback from sensors to adjust behavior on the fly**.
- This is particularly important in dynamic, unstructured environments like orchards, homes, or hospitals, where unexpected changes can alter the task parameters.
- During execution, state tokens are updated in real time, reflecting sensor inputs and joint feedback.
- The model can then revise its planned actions accordingly. This capability mimics human-like adaptability and is a core advantage of VLA systems over pipeline-based robotics.

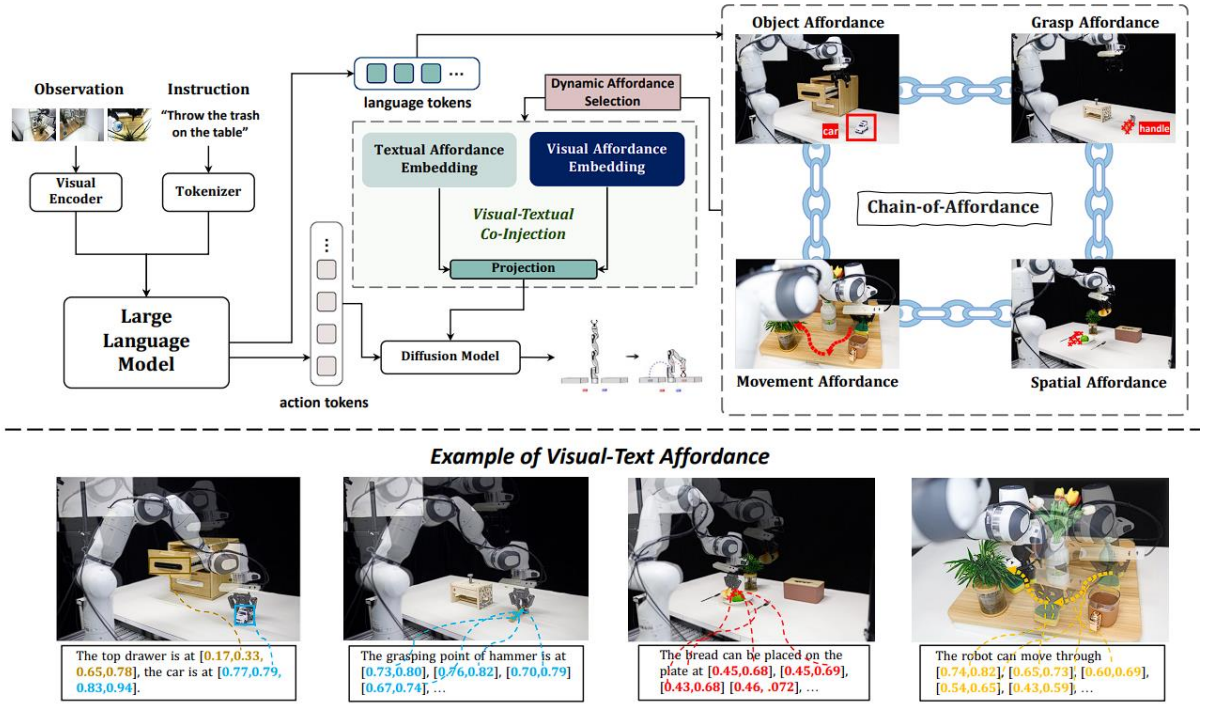


Discussion: E2E vs Modular Pipelines

End-to-end VLAs, such as RT-1, process raw sensory inputs directly into motor commands via a single unified network.

Component-focused models like Chain-of Affordance decouples perception, language grounding, and action modules, enabling targeted improvements in individual submodules.

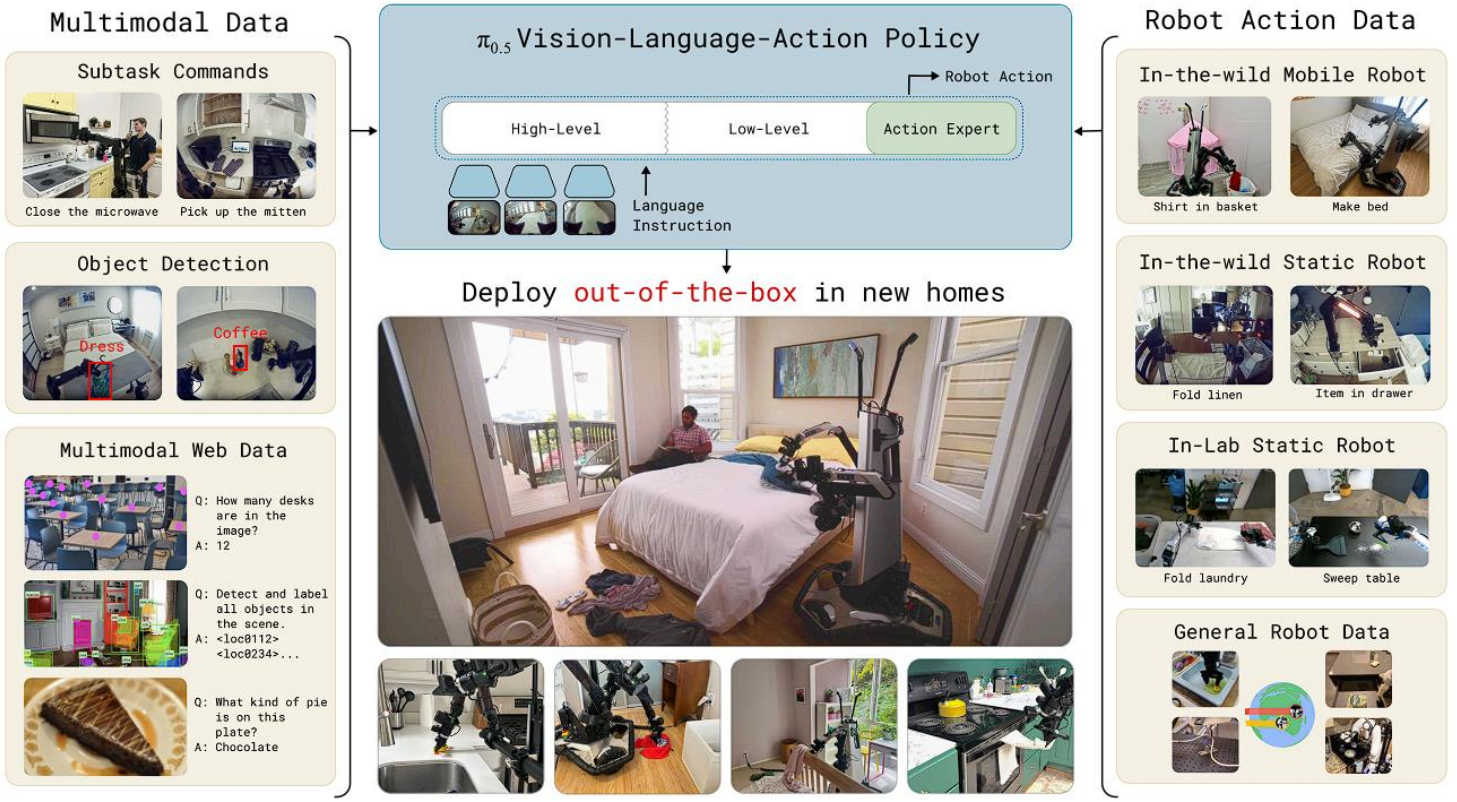




Discussion: Low-level policy & High-level policy

Low-level policy emphasis is typified by diffusion-based controllers (e.g. Pi-0), which excel at producing smooth, diverse motion distributions but often incur higher computational cost.

High-level planners (e.g. FAST Pi-0 Fast, CoVLA) focus on rapid subgoal generation or coarse trajectory prediction, delegating fine-grained control to specialized modules or traditional motion planners



Evaluating VLA Models

Metrics:

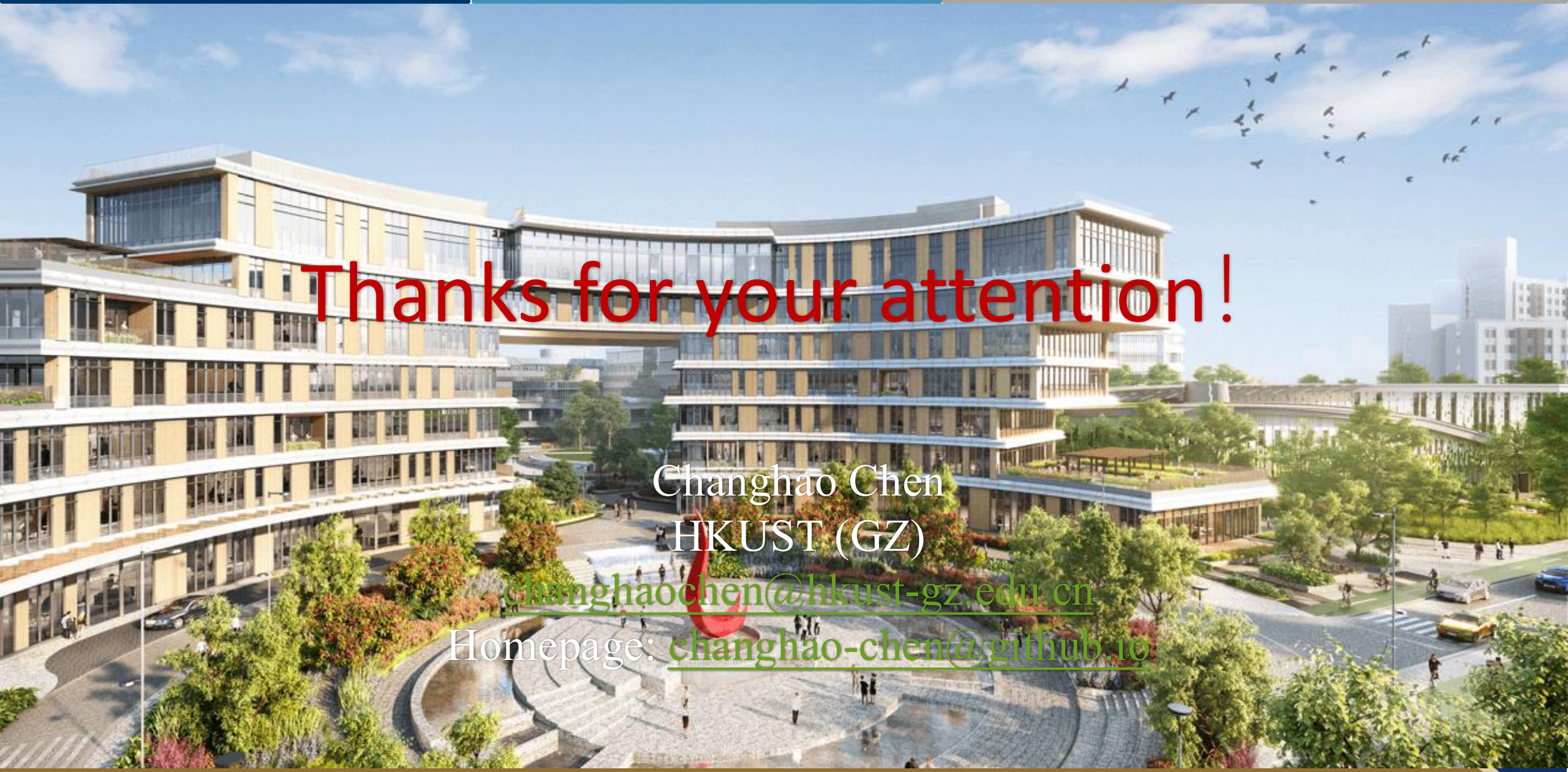
- **Success Rate** (SR)
- **Completion Rate**
- **Goal Distance / Navigation Error**
- **Trajectory Efficiency**
- **Instruction-Following Accuracy**

Generalization tests:

- novel objects
- novel instructions
- novel backgrounds
- novel environments
- long-horizon tasks

Reading Materials

[1] Sapkota R, Cao Y, Roumeliotis K I, et al. Vision-language-action models: Concepts, progress, applications and challenges[J]. arXiv preprint arXiv:2505.04769, 2025.
[2] Helix: A Vision-Language-Action Model for Generalist Humanoid Control https://www.figure.ai/news/helix
[3] Fu H, Zhang D, Zhao Z, et al. Orion: A holistic end-to-end autonomous driving framework by vision-language instructed action generation[J]. arXiv preprint arXiv:2503.19755, 2025.
[4] Cheng A C, Ji Y, Yang Z, et al. Navila: Legged robot vision-language-action model for navigation[J]. arXiv preprint arXiv:2412.04453, 2024.
[5] Fu H, Zhang D, Zhao Z, et al. Orion: A holistic end-to-end autonomous driving framework by vision-language instructed action generation[J]. arXiv preprint arXiv:2503.19755, 2025.
[6] BROHAN A, BROWN N, CHEBOTAR Y, et al. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control[J].
[7]Black K, Brown N, Driess D, et al. π0: A vision-language-action flow model for general robot control. CoRR, abs/2410.24164, 2024. doi: 10.48550[J]. arXiv preprint ARXIV.2410.24164.
[8]Intelligence P, Black K, Brown N, et al. $\pi_ {0.5} $: a Vision-Language-Action Model with Open-World Generalization[J]. arXiv preprint arXiv:2504.16054, 2025.
[9] Robotic Systems Lab: Legged Robotics at ETH Zürich https://youtu.be/3jDoPobFgwA
[10] ZHANG J, WANG K, WANG S, et al. Uni-NaVid: A Video-based Vision-Language-Action Model for Unifying Embodied Navigation Tasks[J]. 2025.
[11] Li J, Zhu Y, Tang Z, et al. Improving vision-language-action models via chain-of-affordance[J]. arXiv preprint arXiv:2412.20451, 2024.



Thanks for your attention!

Changhao Chen
HKUST (GZ)
changhaochen@hkust-gz.edu.cn
Homepage: changhao-chen@github.io